

# AI Resume Screening & Ranking System

SDE Intern Coding Assignment — Technical Project Report

|           |                                                          |
|-----------|----------------------------------------------------------|
| Language  | Python                                                   |
| Interface | Command-line interface (CLI)                             |
| Input     | 50 supplied PDF resumes                                  |
| Output    | Ranked eligible candidates + rejected candidates in JSON |
| Scoring   | Explainable 100-point heuristic model                    |

## Purpose

Build a reliable, explainable batch screening tool that identifies resumes with evidence of both Python and practical AI/LLM/agent work, scores eligible candidates, optionally enriches scores with public GitHub signals, and reports failures without aborting the batch.

Prepared as an implementation and design document

# 1. Problem and Requirements

The assignment provides approximately 50 resumes and asks for a single-run pipeline. The system must extract candidate details, apply hard eligibility gates, score eligible candidates, enrich GitHub profiles when available, and produce machine-readable output with evidence and batch statistics.

## Core requirements implemented

- PDF text extraction with DOCX/TXT support as a bonus.
- Python and AI/Agentic evidence are independent hard eligibility conditions.
- Eligible candidates are ranked by total score; rejected candidates include explicit rejection reasons.
- Scores are broken down into five categories and capped at their specified weights.
- GitHub failures, malformed files, and duplicate files are isolated from the rest of the batch.
- JSON output includes batch statistics, matched skills, evidence, project summary, strengths, concerns, and GitHub status.

# 2. System Architecture

The CLI delegates to a small set of modules with separate responsibilities:

| Module                   | Responsibility                                                            |
|--------------------------|---------------------------------------------------------------------------|
| main.py                  | CLI arguments, invoke pipeline, print batch summary                       |
| src/parser.py            | PDF/DOCX/TXT extraction, text normalization, name/email/GitHub extraction |
| src/screener.py          | Eligibility gates, evidence detection, scoring, ranking, JSON assembly    |
| src/github_enrichment.py | Optional public GitHub repository lookup and failure handling             |
| src/config.py            | Scoring weights and term dictionaries                                     |
| tests/test_screener.py   | Tests for eligibility and score consistency                               |

## Processing flow

Input directory → enumerate resumes → extract text → detect evidence  
→ hard eligibility gate → score eligible candidates → optional GitHub lookup  
→ rank eligible candidates → write results.json + batch summary

# 3. Eligibility Strategy

Eligibility is rule-based and occurs before ranking. A resume must contain evidence of Python (as a skill, project technology, or implementation language) and at least one meaningful AI/LLM/RAG/agentic technology or implementation. JavaScript, Java, React, and Next.js are not disqualifying by themselves. Rejected candidates receive a reason for each missing requirement.

The implementation uses configurable term dictionaries. This is transparent and repeatable, but it can miss synonyms or mistakenly match a term that is mentioned without meaningful implementation context. The output includes matched terms to make decisions auditable.

# 4. Scoring Model

| Category                 | Max | Example signals                                              |
|--------------------------|-----|--------------------------------------------------------------|
| AI / Agentic / RAG depth | 40  | RAG, agents, retrieval, embeddings, tool calling, evaluation |
| Python & backend         | 30  | Python, APIs, FastAPI/Django/Flask, SQL, Redis, async        |

| Category                        | Max | Example signals                                               |
|---------------------------------|-----|---------------------------------------------------------------|
| Cloud / deployment / full stack | 15  | Docker, cloud, deployment, CI/CD, React/Next.js               |
| GitHub activity                 | 10  | Recent repository updates and relevant Python/AI repositories |
| Engineering depth               | 5   | Tests, caching, queues, observability, concurrency, retries   |

The score is deterministic and capped at 100. The current baseline is intentionally dependency-light and does not require an LLM API key. AI project depth is constrained when the project section is missing and penalized when evidence is shallow. This makes the output reproducible, while sacrificing some semantic nuance compared with structured LLM extraction.

## 5. GitHub Enrichment and Reliability

When a GitHub profile is detected, the application requests public repository metadata and estimates activity from repository update timestamps plus the number of Python/AI-relevant repositories. GitHub contributes at most 10 points and is not an eligibility condition. API errors, rate limits, timeouts, and unavailable profiles are recorded as statuses; screening continues.

### Reliability controls

- Each resume is processed inside its own exception boundary.
- Unsupported or unreadable files are recorded in the failures array rather than stopping the batch.
- Exact duplicate file content is detected with SHA-256 and skipped.
- Network calls use a timeout and optional GITHUB\_TOKEN environment variable.
- Configuration and score weights are separate from core pipeline logic.

## 6. Running the Project

```
python -m venv .venv
pip install -r requirements.txt
python main.py --input ./resumes --output ./output/results.json
python -m unittest discover -s tests -v
```

The CLI writes a machine-readable JSON result. A GitHub token is optional; do not commit secrets. The repository includes a .env.example as a reminder of the supported environment variable, but environment variables must be exported in the shell unless a dotenv loader is added.

## 7. Dataset Run Summary

The pipeline was run against the supplied resume archive. Live GitHub enrichment may be unavailable in a restricted environment, in which case each profile is marked with a failure status and receives no GitHub points; this does not stop screening.

| Metric              | Result |
|---------------------|--------|
| Resumes discovered  | 50     |
| Successfully parsed | 50     |
| Eligible            | 32     |
| Rejected            | 18     |
| Failed / duplicate  | 0      |

### Top ranked candidates (automatically extracted)

| Rank | Candidate             | Score  |
|------|-----------------------|--------|
| 1    | Hari Shanker Sharma   | 89/100 |
| 2    | Bengaluru Karnataka   | 87/100 |
| 3    | Sumaiya Sultana Shaik | 83/100 |
| 4    | Kiran Naregal         | 81/100 |
| 5    | Nevil Jasmin Mehta    | 81/100 |

Review the generated results.json before publishing it: it contains personal details extracted from candidate resumes. A public GitHub repository should not expose candidate emails, phone numbers, or other personal information unless publication is explicitly authorized.

## 8. Tests, Limitations, and Future Work

### Automated tests

The included unit tests check that Python + AI evidence is required, eligible candidates receive a score breakdown, score categories sum to the total, and missing GitHub does not fail eligibility. Test execution status is summarized below.

```
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26, in
warm_spreadsheet_runtime_on_startup
File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spreadsheet_runtime
File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in _warm_feature_flows
File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in _warm_drawing_and_image_flows
File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in bring_to_front
File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line 764, in __call__
File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element collection.
test_missing_github_does_not_affect_eligibility (test_screener.ScreeningTests.test_missing_github_does_not_affect_eligibility) ... ok
test_python_and_rag_is_eligible (test_screener.ScreeningTests.test_python_and_rag_is_eligible) ... ok
test_requires_python_and_ai (test_screener.ScreeningTests.test_requires_python_and_ai) ... ok
```

-----  
Ran 3 tests in 0.002s

OK

### Known limitations

- Keyword-based extraction can miss unusual terminology and does not fully understand project quality.
- Scanned PDFs without embedded text may fail without OCR.
- Name extraction is heuristic and may need correction for some resume layouts.
- GitHub repository update timestamps are only a proxy for activity; public event/commit history would be more accurate.
- Scoring should support, not replace, human review in hiring.

### Next improvements

- Add OCR and more robust section-aware resume parsing.
- Add an optional LLM adapter with schema-validated output and cited resume evidence.
- Cache GitHub results and use bounded concurrency.
- Create a labeled evaluation set to measure false positives and false negatives.

## 9. Conclusion

The project implements the required end-to-end screening flow with hard eligibility gates, a transparent weighted scoring model, optional GitHub enrichment, structured JSON output, and per-file failure handling. The principal trade-off is deterministic keyword-based scoring over semantic LLM evaluation: it is easier to reproduce and test, but should be improved and validated before any real hiring use.